

Class 06

Aggregation, Grouping & Multi-Table Queries

◆ 1. Built-in Functions

postgreSQL comes with many **in-built functions** to process and transform data.

String Functions

Function	Description	Example	Output
CONCAT()	Joins two or more strings	SELECT CONCAT(first_name, ' ', last_name)	John Doe
LENGTH()	Returns length of string	SELECT LENGTH('SQL');	3
UPPER()	Converts text to uppercase	SELECT UPPER('sql');	SQL
LOWER()	Converts text to lowercase	SELECT LOWER('HELLO');	hello
SUBSTRING()	Extracts part of string	SELECT SUBSTRING('Database', 1, 4);	Data



Date & Time Functions

Function	Description	Example	Output
NOW()	Returns current date and time	SELECT NOW();	2025-11-03 12:30:00
CURDATE()	Current date only	SELECT CURDATE();	2025-11-03
DATEDIFF()	Difference between two dates	SELECT DATEDIFF('2025-12-31', '2025-01-01');	364
DATE_FORMAT()	Formats date in readable way	SELECT DATE_FORMAT(NOW(), '%d-%M-%Y');	03-November-2025

12
34

Numeric Functions



ROUND()

Description: Rounds a number to a specified number of decimal places.

Example: `SELECT ROUND(12.567, 2);`

Output: 12.57

÷ MOD()

Description: Returns remainder after division.

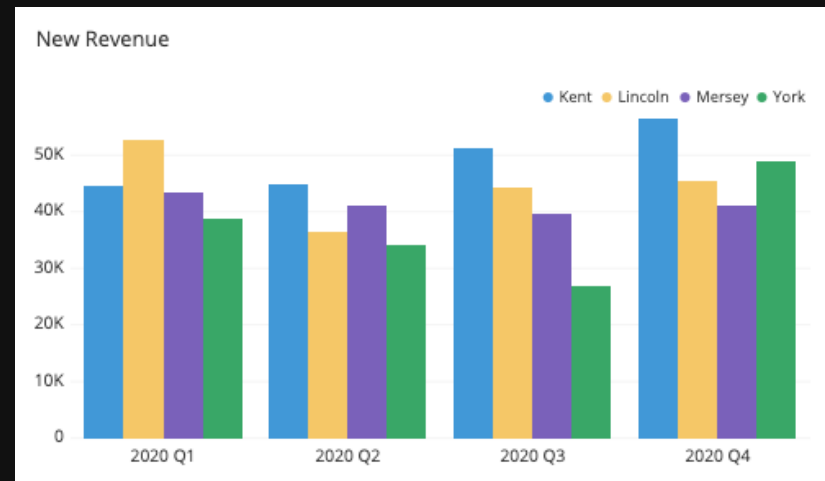
Example: `SELECT MOD(10, 3);`

Output: 1

◆ 2. Aggregate Functions

Aggregate functions perform **calculations on multiple rows** and return a single value.

- **COUNT()** → Counts how many rows exist.
👉 `SELECT COUNT(*) FROM students;`
- **SUM()** → Adds all numeric values.
👉 `SELECT SUM(salary) FROM employees;`
- **AVG()** → Finds the average.
👉 `SELECT AVG(marks) FROM students;`
- **MIN()** → Returns smallest value.
👉 `SELECT MIN(price) FROM products;`
- **MAX()** → Returns largest value.
👉 `SELECT MAX(price) FROM products;`



◆ 3. GROUP BY Clause

The `GROUP BY` clause groups rows that have the **same values** in specified columns and is usually used with **aggregate functions**.

Example:

```
SELECT department, COUNT(*) AS total_employees  
FROM employees  
GROUP BY department;
```

Output:

Department	total_employees
HR	5
IT	8
Sales	6

◆ 4. HAVING Clause

HAVING is used to **filter grouped data**, whereas WHERE filters individual rows.

Example:

```
SELECT department, AVG(salary)
FROM employees
GROUP BY department
HAVING AVG(salary) > 50000;
```

🧠 Note:

- WHERE → filters **before grouping**
- HAVING → filters **after grouping**

WHERE → GROUP BY → HAVING

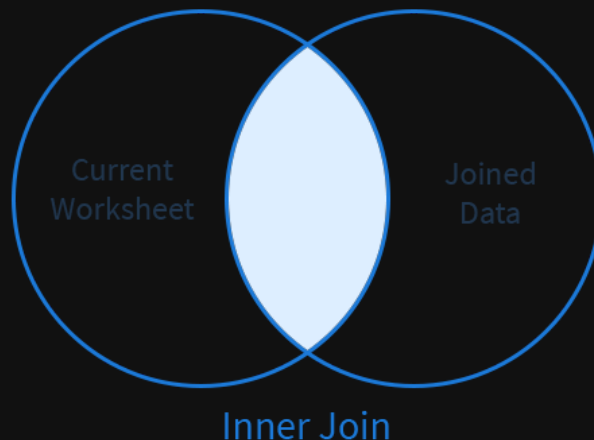
◆ 5. Joins (The Core of postgresQL)

Joins are used to **combine rows from two or more tables** based on related columns.

◆ INNER JOIN

Returns records that have **matching values** in both tables.

```
SELECT students.name, departments.dept_name  
FROM students  
INNER JOIN departments  
ON students.dept_id = departments.id;
```

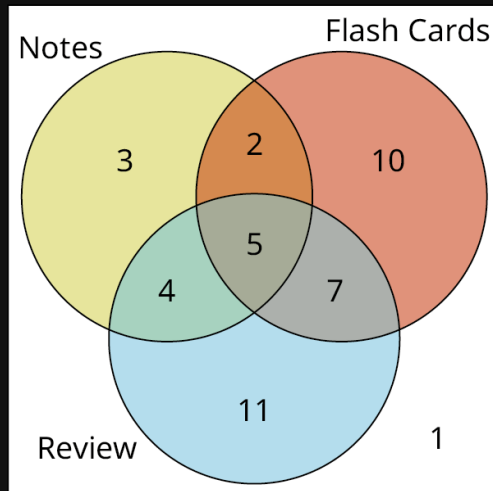


◆ LEFT JOIN (LEFT OUTER JOIN)

Returns all rows from **left table**, and matching rows from the right table.

```
SELECT students.name, departments.dept_name
FROM students
LEFT JOIN departments
ON students.dept_id = departments.id;
```

$U = \text{Class} = 43$



◆ RIGHT JOIN (RIGHT OUTER JOIN)

Returns all rows from **right table**, and matching rows from left.

```
SELECT students.name, departments.dept_name  
FROM students  
RIGHT JOIN departments  
ON students.dept_id = departments.id;
```

◆ FULL JOIN (if supported)

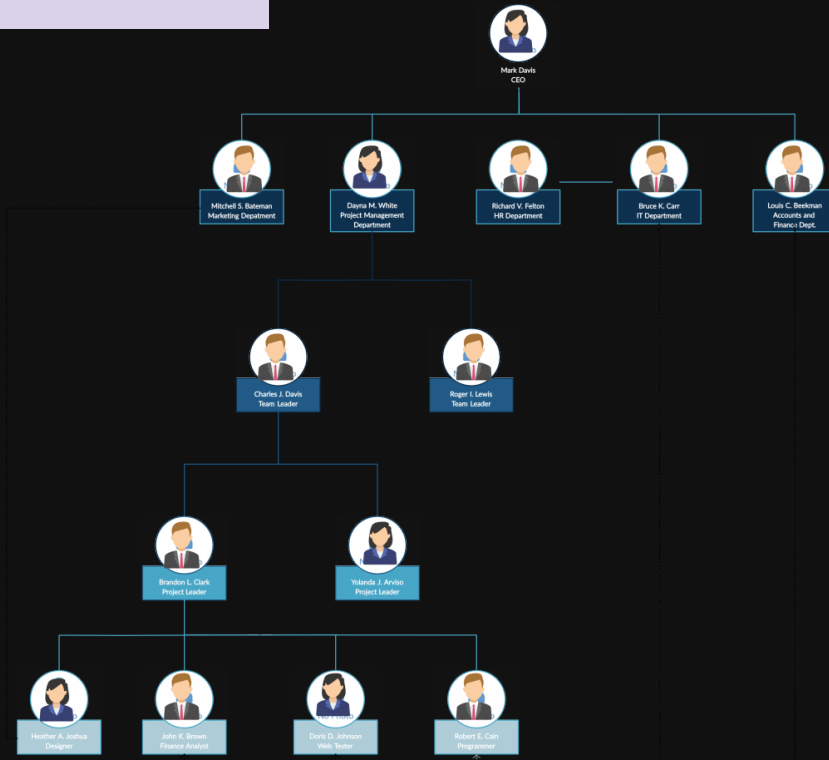
Returns all rows when there is a match in either table.

```
SELECT s.name, d.dept_name  
FROM students s  
FULL JOIN departments d  
ON s.dept_id = d.id;
```

◆ SELF JOIN

Used to join a table with itself (useful for hierarchical data).

```
SELECT e1.name AS Employee, e2.name AS Manager
FROM employees e1
JOIN employees e2
ON e1.manager_id = e2.id;
```



◆ CROSS JOIN

Produces **Cartesian Product** (all combinations of rows).

```
SELECT *  
FROM colors CROSS JOIN sizes;
```

